盾语 • 银行 AI 智能体

技术文档

系统架构 · 核心算法 · 安全设计

Prometheus Ultra | 普罗米修斯 • 极致

项目	内容
作品名称	盾语 · 银行 AI 智能体（仿真环境）
参赛队伍	Prometheus Ultra	普罗米修斯 · 极致
文档版本	v1.0（与提交包 2026-11-01 对齐）
运行形态	Windows 便携包，零 Key 可运行；不连接真实银行
技术栈	Python 标准库 + SQLite WAL + 原生 JS；可选 ONNX/BGE、Vosk

本文件说明系统如何把一句口语变成可审计的业务动作，以及为什么模型不会越权。

1. 定位与边界

盾语是一个面向银行零售场景的 AI 智能体仿真作品。目标不是“接一个大模型”，而是解决三个问题：口语理解、复杂任务编排、以及在高风险金融场景里让模型没有执行权。

- 输入：文字 / 语音 / IM 渠道的一句话，允许口语、省略、错字、复合句。
- 输出：查询结果、方案建议、确认卡、强验证、执行回执、审计链路。
- 边界：所有客户、账户、流水、贷款、信用卡均为合成数据；不连接真实银行核心，不产生真实资金。
- 模型边界：大模型只参与“理解/提议”，不参与授权、不直接调用工具、不生成最终回执。

2. 总体架构



图 1 五层架构：交互、理解、决策、执行、证据

主链路是固定的：消息 → 理解 → 计划 DAG → 七条校验 → 策略三级 → 确认/强验证 → 执行 → 事实账本 → 哈希链审计。任何模型输出都必须经过这条链路，模型没有旁路。

3. 对话处理流水线

阶段	做什么	关键代码
会话/安全前置	注入、角色伪装、密码修改、第三人称查询、否定/咨询	orchestrator.handle_chat
理解	规则 / 语义帧 / BGE / 澄清；模型可选	nlu.py, understanding.py, semantic.py

计划	把意图编译成 DAG；条件任务插入查询节点	planner.py, conditional.py
校验	节点数、环、工具白名单、参数、溯源、单笔/计划上限	validator.py
策略	绿/黄/红三级；返回规则 ID 与理由	policy.py, rules/policy.rego
确认/强验证	确认卡 token/hash/过期；红色确认卡→MFA	orchestrator.py, trust.py
执行	拓扑序执行、幂等键、失败逆序补偿	executor.py, tools.py
证据	事实账本、发出前校验、哈希链审计	trust.py, audit

4. 理解层：分层收敛，不硬猜

理解层按确定性从高到低逐层收敛，任何一层失败都可以回退，不允许“猜一个就执行”。

层	输入	输出	约束
规则	高频说法/关键词	意图 + 槽位	可解释、可回归
语义帧	时间、金额、对象、动作	结构化槽位	金额必须换算为分
BGE 向量	规则未命中的短句	候选意图 + 相似度	≥0.75 直接；0.45–0.75 复述确认；<0.45 澄清
大模型（可选）	自然语言	候选意图/计划	无 Key 回退；非法 JSON/表外工具回退

安全前置先于理解：咨询句（“提前还款有违约金吗”）、否定句（“我不想还房贷”）、第三人称查询（“查张三的余额”）不会生成办理计划。

5. 条件 DAG 与执行



图 2 条件任务的 4 节点计划

条件任务不是“先算完再假装有计划”，而是把查询节点 n0 写进 plan，动作节点 n3 带 condition 与 depends_on。条件成立才出确认卡；不成立直接回答原因，不生成转账卡；有否则分支时执行 else。

```
n3.condition = [{"node": "n0", "field": "total_cents", "op": ">", "value": 100000}]
n3.depends_on = ["n1", "n2", "n0"]
```

支持范围：本月/上月支出、余额、比较符、and、else。明确不支持任意嵌套布尔脚本；条件求值只读，不产生资金动作。

6. 权限与安全设计

级别	范围	控制
绿	余额/流水/账单/持仓/咨询/提醒	直接执行，无确认
黄	日累计 ≤1,000 转账（新收款人单笔 ≤500）、退订、调额、风险测评、虚拟卡	确认卡：金额、对象、后果、params_hash、token、倒计时
红	日累计 >1,000 转账 / 新收款人 >500、挂失、解挂、赎回、还款、密码重置链接	还款/信用卡/赎回：确认卡→MFA；其余红色：摘要→MFA；密码修改恒拒绝

确认卡不是一张图片：token、intent_id、params_hash、过期时间、一次性使用共同绑定。改金额触发 E_PARAM，改 token 触发 E_CONFIRM_TOKEN，执行前重新跑策略，审计链可校验。

七条硬校验（validator.py）：节点数、无环、工具白名单、必填参数/类型、资金参数溯源、单笔 5 万、计划净流出 5 万。任何一条不过就不进入执行。

审计哈希链：每条事件 prev_hash 串联，篡改任意一行会导致后续校验失败；安全证据面板在隔离临时库现场演示 token/金额/审计三种篡改。

失败补偿：执行器按逆序调用 compensate；生日计划注入 create_standing_plan 失败时，已冻结预算原路释放。

熔断：连续失败 3 次 OBSERVE、5 次 LOCKED 15 分钟；密码修改、注入、越权查询前置拒绝。

7. 账实一致与数据模型

对象	数据来源	一致性规则
贷款进度	loans.principal/remaining/monthly/next_due	已还期数按金额差推算；还款后立即变化
信用卡账单	cards.bill_cents + 本月消费	账单/最低还款/额度/已用/可用同源
可用余额	总余额 − 冻结 − active freezes	显示总余额与冻结明细
订阅	subscriptions.status	在用/已取消分组，已取消不计入未使用建议

积分	points 表	演示环境明确不支持兑换

原则：页面上出现的每个数字，都必须能在数据库或工具结果里找到来源；不允许模板写死。

8. 评测与证据

项目	结果	说明
单元/集成测试	448 项：最终便携版 448 全通过	开发机 447 通过 / 1 条环境相关跳过
评测集	243 / 243	意图、权限、红队、端到端
红队注入	58 / 58	端到端拦截
权限边界	60 / 60	三级矩阵与拒绝路径
盲测	27 / 30	公开 3 条失败，不回头调规则
已知绕过	3 条	检测层漏检、执行层全部拦截

复跑命令：python -m unittest discover tests -q；python -m eval.runner；python -m eval.blind_runner。

9. 部署与复跑

- 便携包：解压后双击 start.bat；Python 3.12 embedded 运行时，无需安装。
- 开发：cd backend; python -m bankagent.api；默认 http://127.0.0.1:8000。
- LLM：设置 → 对话 → API Key，保存并启用；也可用环境变量 LLM_API_KEY；失败自动回退规则。
- 演示时钟：默认固定 2026-10-31（与种子数据同月）；需要真实时间设置 BANK_REAL_CLOCK=1。

10. 已知边界

不把没做的说成做了：

- 通用条件 DAG 只支持支出/余额 + 比较 + and + else，不支持任意嵌套/循环/跨用户条件。
- 只有还款、信用卡还款、赎回做了“确认卡→MFA”双闸；其他红色仍是摘要→MFA。
- 人脸为模拟因子，U 盾未实现；生产需替换 StepUpAuthProvider。
- 知识库支持新建/审批/发布，但前端运营页仍以改为主；文档已标注。
- 不连接真实银行核心，不产生真实资金。